

# Topological Quantum Compiling with Reinforcement Learning

Yuan-Hang Zhang,<sup>1,2</sup> Pei-Lin Zheng,<sup>3,4</sup> Yi Zhang,<sup>3,4,\*</sup> and Dong-Ling Deng<sup>1,5,†</sup>

<sup>1</sup>Center for Quantum Information, IIIS, Tsinghua University, Beijing 100084, People's Republic of China

<sup>2</sup>Department of Physics, University of California, San Diego, CA 92093, USA

<sup>3</sup>International Center for Quantum Materials, Peking University, Beijing 100871, China

<sup>4</sup>School of Physics, Peking University, Beijing 100871, China

<sup>5</sup>Shanghai Qi Zhi Institute, 41th Floor, AI Tower, No. 701 Yunjin Road, Xuhui District, Shanghai 200232, China

(Dated: October 22, 2020)

Quantum compiling, a process that decomposes the quantum algorithm into a series of hardware-compatible commands or elementary gates, is of fundamental importance for quantum computing. We introduce an efficient algorithm based on deep reinforcement learning that compiles **an arbitrary single-qubit gate** into a sequence of elementary gates from a finite universal set. It generates near-optimal gate sequences with given accuracy and is generally applicable to various scenarios, independent of the hardware-feasible universal set and free from using ancillary qubits. For concreteness, we apply this algorithm to the case of topological compiling of Fibonacci anyons and obtain near-optimal braiding sequences for arbitrary single-qubit unitaries. Our algorithm may carry over to other challenging quantum discrete problems, thus open up a new avenue for intriguing applications of deep learning in quantum physics.

To efficiently decompose unitaries into a sequence of elementary hardware-compatible quantum gates as short as possible is a crucial problem in a variety of quantum-information-processing tasks, such as quantum computing [1] and quantum digital simulations [2]. This problem becomes especially relevant for the noisy intermediate-scale quantum devices [3], where the depth of the quantum circuits might be limited due to the inaccuracy of the possible elementary gates and quantum decoherence. A number of notable algorithms have been proposed to compile single-qubit unitaries [4–17]. For instance, the Solovay-Kitaev algorithm runs in  $O(\log^{2.71}(1/\epsilon))$  time and can output a sequence of  $O(\log^{3.97}(1/\epsilon))$  elementary gates that approximate the targeted unitary to precision  $\epsilon$  [1, 4]. Other algorithms either exploit the specific structure of the Clifford+ $T$  gate set [12–16], or utilize ancillary qubits [5, 6], to further reduces the running time and length of the desired gate sequences. Each of these algorithms bears its pros and cons, and the choice depends on the specific problem. Here, inspired by the similarity between quantum compiling and solving Rubik's cube (see Table I), we introduce a novel algorithm based on deep reinforcement learning, which compiles single-qubit unitaries efficiently and is generally applicable to different scenarios (see Fig. 1 for an illustration).

Machine learning, especially deep learning, has achieved dramatic success in a broad range of artificial intelligence applications, ranging from image/speech recognition to self-driving cars [19, 20]. The interplay between machine learning and quantum physics has led to an emergent research frontier of quantum machine learning, which has attracted tremendous attention [21–24]. Quantum learning algorithms with potential exponential advantages have been proposed, and machine learning techniques have also been invoked in various applications in quantum physics, including representing quantum many-body states [25, 26], quantum state tomography [27, 28], non-locality detection [29], and learning phases of matter [30–40], etc. In this work, we introduce deep reinforcement learning [41], which has been exploited to

| System        |                                |                      |
|---------------|--------------------------------|----------------------|
| Initial state | The unitary to be approximated | The scrambled cube   |
| Target state  | The identity matrix            | The solved cube      |
| Basic move    | A gate from the universal set  | Rotation of one face |

TABLE I. Comparison between quantum compiling and solving the Rubik's cube shows that these two seemingly irrelevant problems have a lot in common. The shown cube only has two misplaced cubelets, yet it takes at least 16 steps [18] to solve. Similarly, the shown quantum state  $|\psi\rangle$  is close to the target state  $|0\rangle$ , but with a discretized universal gate set, it may still take many steps to transform into  $|0\rangle$ .

build AlphaGo [42] (a computer program of Go that defeated the world's best players) and more recently DeepCubeA that solves the Rubik's cube — a classic combinatorial puzzle that posed unique challenges for artificial intelligence [43] to the task of quantum compiling. We observe that compiling unitaries to a sequence of elementary gates is analogous to finding a sequence of basic moves that solves the Rubik's cube (see Table I). Since unitaries are invertible, finding a gate sequence approximating a target unitary  $U$  is equivalent to finding a gate sequence that “restores”  $U$  back to identity. In this way, the identity matrix becomes our target state (corresponding to the solved cube), and the unitary  $U$  is the initial state (corresponding to a scrambled cube). Both problems have several discretized, non-commuting operations; the goal of both problems is to find the shortest sequences available; for a state seemingly close to the targeted one, the actual number

of required operations may still be surprisingly large. Similar to the fact that DeepCubeA can solve an arbitrarily scrambled cube in a near-optimal fashion [43], our algorithm can efficiently compile an arbitrary unitary into a near-optimal sequence of elementary gates.

*The algorithm*—First, we introduce our general algorithm; later, we will apply it to the case of topological compiling of Fibonacci anyons as a concrete example. In previous reinforcement learning algorithms such as deep Q-learning [42, 44], a function approximator such as a deep neural network (DNN) represents a reward function defined on all states, which dictates the strategy to maximize the reward and performs the actions step-by-step. Then, the resulting experiences are added to the regression to optimize the DNN further, and so on so forth. However, when such an algorithm is directly applied to bring an arbitrary quantum state to a specific target, it faces immediate failure: with a large state space, discretized actions at each step, a single target, and only giving the reward extremely close to the target, the reward may never be received at all, making it almost impossible to train a valid reward function.

To resolve this issue, we start from the target state instead and perform backward search operations, similar to the value iteration algorithm [45]. The cost-to-go function  $J(s)$  is defined as the minimum cost for a state  $s$  to reach the target state within the designated precision, represented approximately by a DNN. During training, we update the cost-to-go function according to [43]:

$$J'(s) = \min_a (g(s, a) + J(S(s, a))) \quad (1)$$

where  $S(s, a)$  is the state obtained after applying the action  $a$  to the state  $s$ , and  $g(s, a)$  is the corresponding cost.  $J(s_0) = 0$  for the target state  $s_0$ , and  $J(s)$  for other states can be computed with Eq. (1) successively. In practice, Eq. (1) uses the DNN itself for target updating, which may lead to instabilities. Therefore, we use two neural networks during training [43, 44]: a policy network that is constantly being trained, and a target network that estimates of the target value  $J'(s)$  for training and only updates to the policy network periodically.

To enhance the search performance and derive the shortest sequence possible, we further complement the cost-to-go function  $J(s)$  with a weighted  $A^*$  search algorithm [46, 47]. We define an evaluation function  $f(s)$  from the initial state  $s_i$  to the target state  $s_0$  via an intermediate state  $s$ :

$$f(s) = \lambda G(s) + J(s) \quad (2)$$

where  $G(s)$  is the actual cost from the initial state  $s_i$  to the current state  $s$ .  $\lambda \in [0, 1]$  is a weighting factor, and smaller  $\lambda$  reduces the number of states evaluated and alleviates the difficulty of a large state space at the expense of potentially longer paths [47]. During the search, we start with a set of the intermediate states  $\{s\}$  with only the initial state  $s_i$ ; iteratively, we pick the state  $s$  in  $\{s\}$  with the minimum  $f(s)$  and replace it with its successors  $S(s, a)$  (if they are not already in or have not previously been in  $\{s\}$ ), see Fig. 1; once a state with a

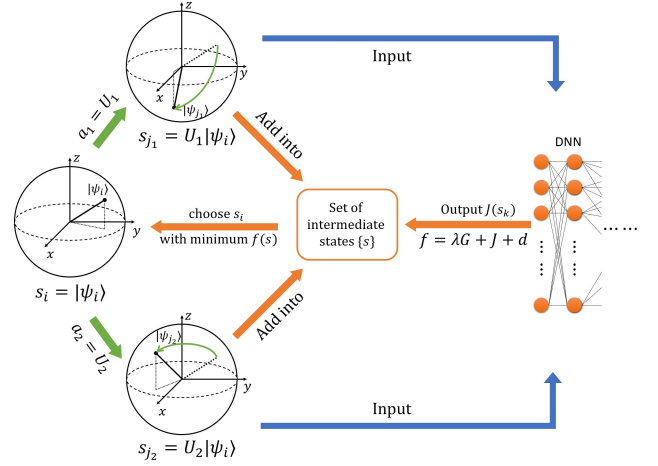


FIG. 1. A schematic illustration of the algorithm: during the search, we start from the initial state  $s_i = |\psi_i\rangle$ , then iteratively pick the state with the minimum evaluation function  $f(s)$  and evaluate all its successors with the DNN until we reach the target state (see [48]).

distance less than a designated termination accuracy  $\epsilon_T$  from the target state  $s_0$  is present in  $\{s\}$ , we have obtained the desired sequence between  $s_i$  and  $s_0$  within the desired accuracy threshold.

We also make several additional modifications to the weighted  $A^*$  search algorithm to better fit our quantum compiling problem. First, we introduce a maximum searching depth  $D_{\max}$ , beyond which the search terminates and returns the best state found so far. This cut off resolves the possible non-convergence induced by the search along a discrete graph on a continuous state space. Second, it is natural for the DNN to generalize the cost-to-go function  $J(s)$  to states never present in training. Sometimes, such a state is mistaken for a small  $J(s)$  (e.g.,  $J(s) \sim 1.5$ ), although its actual distance from the target state is considerably farther away, and the weighted  $A^*$  searches are stuck there. To handle this problem, we introduce a decimal-penalty term to the evaluation function  $f(s) = \lambda G(s) + J(s) + d(s)$ :

$$d(s) = \gamma \frac{[J(s) - \text{round}(J(s))]^2}{J(s)} \quad (3)$$

where  $\gamma$  is a constant tuning parameter.  $d(s)$  put preferences on states used to train the DNN with near-integer  $J(s)$  over states whose  $J(s)$  values containing decimal parts and are likely estimations and interpolations.

Without loss of generality, we apply our algorithm to topological compiling with Fibonacci anyons, which are quasiparticle excitations of topological states that obey non-Abelian braiding statistics [49]. Unlike Majorana bound states [50], whose braiding only gives elementary gates in the Clifford group unless additional multi-step protocols are incorporated [51], Fibonacci anyons are the simplest non-Abelian quasiparticles that enable universal topological quantum computation [52, 53] by braiding alone [54]. They are theoretically predicted to exist in the  $\nu = 12/5$  fractional quantum Hall

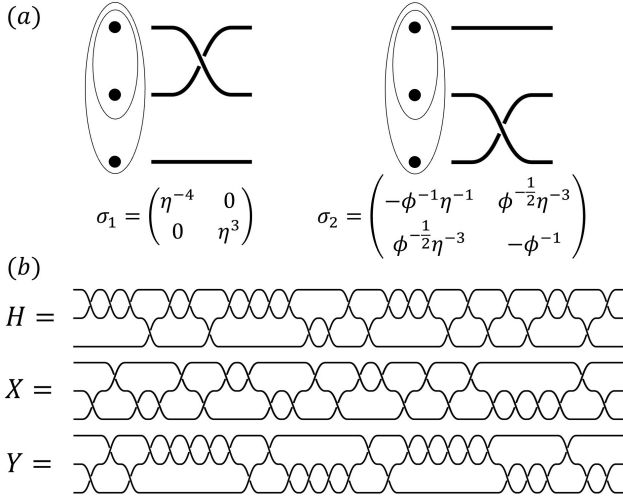


FIG. 2. (a) The two elementary gates by braiding Fibonacci anyons. Logical qubits are encoded into triplets of anyons (enclosed in the ovals), and time flows from left to right. Here,  $\eta = e^{i\pi/5}$  and  $\phi = \frac{\sqrt{5}+1}{2}$  [49]. (b) The approximate braiding sequences obtained by the reinforcement-learning based algorithm for the Hadamard gate (H), the Pauli  $\sigma^x$  gate (X), and the Pauli  $\sigma^y$  gate (Y), respectively. The quaternion distances between the braiding sequences and their corresponding targets are  $4.4 \times 10^{-3}$ ,  $2.4 \times 10^{-3}$ , and  $2.3 \times 10^{-3}$ , respectively. After the DNN is trained, the algorithm takes only a couple of seconds to output each of the sequences.

liquid [55] and rotating Bose condensates [56], as well as quantum spin systems [57, 58]. The only nontrivial fusion rule for Fibonacci anyons reads  $\tau \times \tau = \mathbf{I} + \tau$ , where  $\mathbf{I}$  and  $\tau$  denote the vacuum and the Fibonacci anyon, respectively. We encode logical qubits into triplets of anyons with total topological charge one [54]:  $|0_L\rangle = |((\bullet, \bullet)_\tau, \bullet)_\tau\rangle$  and  $|1_L\rangle = |((\bullet, \bullet)_\tau, \bullet)_\tau\rangle$ , and neglect the non-computational state  $|NC\rangle = |((\bullet, \bullet)_\tau, \bullet)_\tau\rangle$  since we mainly focus on braidings within a single logical qubit and the leakage error is not relevant in this case. Based on this encoding scheme, the two elementary single-qubit gates correspond to the braidings of two Fibonacci anyons  $\sigma_1$  and  $\sigma_2$  as shown in Fig. 2(a), which form a universal set for single-qubit unitaries.

In the literature, topological compiling with Fibonacci anyons has been extensively studied, and different algorithms have been proposed [59–65]. Notable examples include the quantum hashing algorithm [63], which runs in  $O(\log(1/\epsilon))$  time and output a sequence of length  $O(\log^2(1/\epsilon))$ , and the probabilistically polynomial algorithm [64], which runs in  $O(\text{polylog}(1/\epsilon))$  time on average and outputs an asymptotically depth-optimal sequence of length  $O(\log(1/\epsilon))$ . Here we apply the introduced reinforcement-learning algorithm. To measure the accuracy of the output sequence, we use the quaternion distance [66]:  $d(q_b, q_t) = \sqrt{1 - \langle q_b, q_t \rangle^2}$ , where  $q_b$  and  $q_t$  are the unit quaternions corresponding to the unitary from the braiding sequence and the target unitary respectively, and  $\langle q_b, q_t \rangle$  denotes their inner product. We employ a DNN with the state  $s$  as the input, two fully-connected hidden lay-

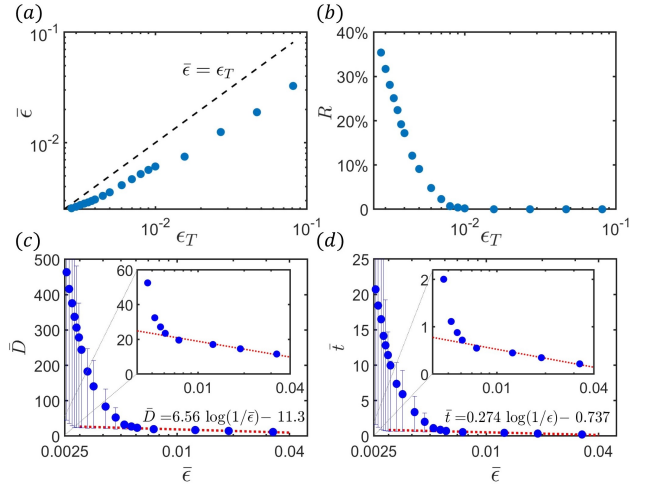


FIG. 3. (a) The averaged actual accuracy ( $\bar{\epsilon}$ ) versus the termination accuracy ( $\epsilon_T$ ). Since the distribution of the accuracy has a long tail, we employ the typical average  $\bar{\epsilon} = \exp(\log \epsilon_i)$ . Here, the weighted A\* search terminates once a state with an accuracy smaller than  $\epsilon_T$  is found, or the searching depth exceeds  $D_{\max} = 1000$ . (b) The ratio ( $R$ ) of target unitaries that require a depth larger than  $D_{\max}$  to attain an accuracy smaller than  $\epsilon_T$ . In general, we need longer sequences for higher accuracy. (c) The average searching depth ( $\bar{D}$ ) as a function of the average actual accuracy ( $\bar{\epsilon}$ ). The red dotted line is a logarithmic fitting of  $\bar{D}$  into  $\log(1/\bar{\epsilon})$ . (d) The average searching time  $\bar{t}$  versus the average actual accuracy  $\bar{\epsilon}$ . The red dotted line is a logarithmic fitting of  $\bar{t}$  into  $\log(1/\bar{\epsilon})$ . In these figures, each data point represents an average on the solutions of 1000 random unitaries.

ers, and six residue blocks [67], followed by one output neuron representing the approximate cost-to-go function  $J(s)$ . We train this DNN via PyTorch routines with randomly sampled sequences whose lengths are shorter than a given constant [48]. The training process takes about two days running on a NVIDIA TITAN V GPU. Without loss of generality, we set  $g(s, a) = 1$  for all gates in Eq. (1). In situations where certain elementary gates are harder to implement, or the cost is state-dependent, we can simply adjust  $g(s, a)$  and retrain the DNN. The optimal values for parameters  $\lambda, \gamma$  in the evaluation function  $f(s)$  and the maximum searching depth  $D_{\max}$  are determined by a grid search (see [48]). Unless noted otherwise, we set  $\lambda = 1$ ,  $\gamma = 400$ , and  $D_{\max} = 100$ .

In Fig. 2(b), we show the braiding sequences derived by our reinforcement-learning algorithm to approximate the Hadamard gate, the Pauli  $\sigma^x$  gate, and the Pauli  $\sigma^y$  gate (up to a trivial global phase). Compare to the brute-force searches in previous works [59, 61, 62], we have achieved comparable accuracy and sequence length, but with much less computational time. Also, we randomly generate 1000 unitaries in  $SU(2)$  and use the reinforcement-learning algorithm to generate their corresponding braiding sequences. The algorithm can efficiently output the desired sequence for any of these unitaries with a running time of less than a couple of seconds on a single GPU. The typical average length of these sequences is  $\sim 24.79$ , and the average precision is  $\sim 3.1 \times 10^{-3}$

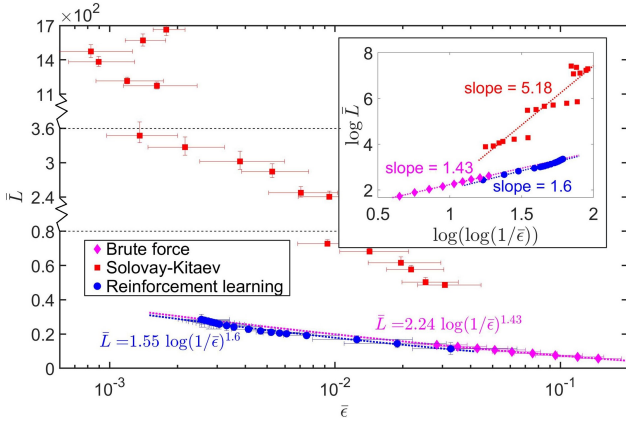


FIG. 4. Comparison between different algorithms on the length complexity of the generated gate sequences. Each data point represents an average on the solutions of 1000 random unitaries, and the error bars show the interval between the lower 25th percentile and the upper 75th percentile. Note the breaks in the vertical scale. The inset is a log-log plot for  $\bar{L}$  versus  $\log(1/\bar{\epsilon})$ , and the dotted lines are linear fits of the data.

(see [48]), on par with the results from the brute-force search. To compare our algorithm with the Solovay-Kitaev algorithm, we apply the latter to the same 1000 unitaries and find the obtained braiding sequences are typically ten times longer.

To further analyze the time complexity and the length complexity as the scalings of the precision inverse  $1/\epsilon$ , we explicitly control the approximation accuracy by terminating the weighted  $A^*$  search once a state with a distance less than  $\epsilon_T$  from the target state  $s_0$  is found. To ensure that most instances reach the desired accuracy  $\epsilon_T$ , here we set the maximum searching depth to a larger value  $D_{\max} = 1000$ . Fig. 3(a) shows the averaged actual accuracy  $\bar{\epsilon}$  as a function of  $\epsilon_T$ . When  $\epsilon_T$  is large, it is easier to find a sequence with a precision smaller than  $\epsilon_T$ , and the search terminate before hitting the depth limit  $D_{\max}$ , thus  $\bar{\epsilon}$  is noticeable smaller than  $\epsilon_T$ . As  $\epsilon_T$  becomes smaller, the constraint of limited searching depth becomes dominant, and more and more target unitaries may require the weighted  $A^*$  searches with a depth larger than the given  $D_{\max} = 1000$  to attain an accuracy smaller than  $\epsilon_T$ , as shown in Fig. 3(b). We plot the averaged searching depth  $\bar{D}$  as a function of  $\bar{\epsilon}$  in Fig. 3(c). From this figure, when  $\bar{\epsilon}$  is large,  $\bar{D}$  scales logarithmically with  $1/\bar{\epsilon}$ :  $\bar{D} \sim 6.56 \log(1/\bar{\epsilon})$ , leading to a nearly linear time complexity — the search time scales as  $\bar{t} \sim 0.274 \log(1/\bar{\epsilon})$ , see Fig. 3(d). As  $\bar{\epsilon}$  decreases further, however, the searching depth and time start to increase dramatically. This is likely due to the relatively limited sequence length (no larger than  $D \sim 40$ ) during the training, thus the DNN has not yet learned enough information for approximating unitaries with higher precision. One way to improve the performance of the algorithm for smaller  $\bar{\epsilon}$  is to increase  $D_{\max}$ . Also, we plot the average length  $\bar{L}$  of the braiding sequences obtained by different algorithms as a function of  $\bar{\epsilon}$  in Fig. 4. From this figure,  $\bar{L}$  scales as  $\bar{L} \sim \log^{1.6}(1/\bar{\epsilon})$  for our reinforcement-learning algorithm, which is slightly worse

than the scaling for the brute-force approach but notably better than that for the Solovay-Kitaev algorithm. We note that one may further improve the performance of the reinforcement-learning algorithm in the above example, through increasing the size of the DNN, the length of the braiding sequences in the training set, or the searching depth when generating sequences, etc. In fact, we used a much smaller DNN in this work than that for AlphaGo [42] and only a single GPU.

The reinforcement learning algorithm can also compile two- or multi-qubit gates, with enlarged state space (target unitary matrices) and action space (gates in the universal set) accordingly, which demands a larger DNN, and inevitably, increases the cost for its training. For simplicity, here we consider the compiling of arbitrary two-qubit gates for demonstration. The action space involves braiding six Fibonacci anyons within the 87-dimensional Hilbert space, much larger than the case for single-qubit gates [59]. Alternatively, we can decompose an arbitrary two-qubit gate into seven single-qubit gates and three controlled-NOT (CNOT) gates analytically and optimally [68]. In turn, the CNOT gate can be decomposed into a single-qubit rotation and a controlled- $iX$  gate, whose braiding sequence is available [59, 61]. Finally, our reinforcement learning algorithm can compile the component single-qubit unitaries. Indeed, this build-up decomposes arbitrary two-qubit gates into braiding sequences with notably better performance than the Solovay-Kitaev algorithm [48].

*Discussion and conclusion*—In experiments, it is common that elementary gates cost differently, and reducing the use of the expensive ones in compiling is of crucial importance for applications in quantum computing. Notably, each elementary gate's cost can be naturally incorporated into our reinforcement-learning approach by adjusting the cost function  $[g(s, a)]$  in Eq. (1) — another striking advantage of the proposed approach over traditional algorithms. Moreover, our approach carries over straightforwardly to other quantum control problems [69] as well.

In summary, we have introduced a reinforcement-learning-based quantum compiling algorithm to decompose an arbitrary unitary into a sequence of elementary gates from a finite universal set. This algorithm uses no ancillary qubit or group-theory relevance and is generally applicable to various scenarios regardless of the choice of universal gate sets. It generates near-optimal gate sequences that approximate arbitrary unitaries to given accuracy in an efficient fashion. To illustrate how the algorithm works, we have also applied it to topological compiling with Fibonacci anyons. Our results build a new connection between reinforcement learning and quantum compiling, which would benefit future studies in both areas.

*Acknowledgment*—We acknowledge insightful discussions with Lei Wang. Y.Z. and P.L.Z. are supported by the start-up grant from Peking University. Y.H.Z. and D.L.D. acknowledge the start-up fund from Tsinghua University (Grant. No. 53330300320). DLD also would like to acknowledge additional support from the Shanghai Qi Zhi Institute. The source code for this work can be found at [https://github.com/yuanhangzhang98/ml\\_quantum\\_compiling](https://github.com/yuanhangzhang98/ml_quantum_compiling).

\* frankzhangyi@gmail.com

† dldeng@tsinghua.edu.cn

- [1] M. A. Nielsen and I. Chuang, “Quantum computation and quantum information,” (2002).
- [2] I. M. Georgescu, S. Ashhab, and F. Nori, “Quantum simulation,” *Rev. Mod. Phys.* **86**, 153 (2014).
- [3] J. Preskill, “Quantum Computing in the NISQ era and beyond,” *Quantum* **2**, 79 (2018).
- [4] C. M. Dawson and M. A. Nielsen, “The solovay-kitaev algorithm,” *arXiv preprint quant-ph/0505030* (2005).
- [5] A. Y. Kitaev, A. H. Shen, and M. N. Vyalyi, *Classical and Quantum Computation* (American Mathematical Society, 2002).
- [6] N. C. Jones, J. D. Whitfield, P. L. McMahon, M.-H. Yung, R. V. Meter, A. Aspuru-Guzik, and Y. Yamamoto, “Faster quantum chemistry simulation on fault-tolerant quantum computers,” *New Journal of Physics* **14**, 115023 (2012).
- [7] A. G. Fowler, “Constructing arbitrary steane code single logical qubit fault-tolerant gates,” *Quantum Info. Comput.* **11**, 867–873 (2011).
- [8] A. Bocharov and K. M. Svore, “Resource-optimal single-qubit quantum circuits,” *Phys. Rev. Lett.* **109**, 190501 (2012).
- [9] A. Bocharov, Y. Gurevich, and K. M. Svore, “Efficient decomposition of single-qubit gates into  $v$  basis circuits,” *Phys. Rev. A* **88**, 012313 (2013).
- [10] T. T. Pham, R. Van Meter, and C. Horsman, “Optimization of the solovay-kitaev algorithm,” *Phys. Rev. A* **87**, 052332 (2013).
- [11] Y. Zhiyebayev, V. M. Akulin, and A. Mandilara, “Quantum compiling with diffusive sets of gates,” *Phys. Rev. A* **98**, 012325 (2018).
- [12] V. Kliuchnikov, D. Maslov, and M. Mosca, “Asymptotically optimal approximation of single qubit unitaries by clifford and  $t$  circuits using a constant number of ancillary qubits,” *Physical review letters* **110**, 190502 (2013).
- [13] P. Selinger, “Quantum circuits of  $t$ -depth one,” *Phys. Rev. A* **87**, 042302 (2013).
- [14] D. Gosset, V. Kliuchnikov, M. Mosca, and V. Russo, “An algorithm for the  $t$ -count,” *Quantum Info. Comput.* **14**, 1261–1276 (2014).
- [15] R. N. J. and P. Selinger, “Optimal ancilla-free clifford+ $t$  approximation of  $z$ -rotations,” *Quantum Info. Comput.* **16**, 901 (2016).
- [16] L. E. Heyfron and E. T. Campbell, “An efficient quantum compiler that reduces  $t$  count,” *Quantum Science and Technology* **4**, 015004 (2018).
- [17] M. Sohaib Alam, “Quantum Logic Gate Synthesis as a Markov Decision Process,” *arXiv e-prints*, arXiv:1912.12002 (2019), [arXiv:1912.12002 \[quant-ph\]](#).
- [18] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, “The diameter of the rubik’s cube group is twenty,” *SIAM Review* **56**, 645 (2014).
- [19] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature* **521**, 436 (2015).
- [20] M. Jordan and T. Mitchell, “Machine learning: Trends, perspectives, and prospects,” *Science* **349**, 255 (2015).
- [21] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature* **549**, 195 (2017).
- [22] S. D. Sarma, D.-L. Deng, and L.-M. Duan, “Machine learning meets quantum physics,” *Physics Today* **72**, 48 (2019).
- [23] V. Dunjko and H. J. Briegel, “Machine learning & artificial intelligence in the quantum domain: a review of recent progress,” *Rep. Prog. Phys.* **81**, 074001 (2018).
- [24] G. Carleo, I. Cirac, K. Cranmer, L. Daudet, M. Schuld, N. Tishby, L. Vogt-Maranto, and L. Zdeborová, “Machine learning and the physical sciences,” *Rev. Mod. Phys.* **91**, 045002 (2019).
- [25] G. Carleo and M. Troyer, “Solving the quantum many-body problem with artificial neural networks,” *Science* **355**, 602 (2017).
- [26] X. Gao, Z. Zhang, and L.-M. Duan, “An efficient quantum algorithm for generative machine learning,” *arXiv:1711.02038* (2017).
- [27] G. Torlai, G. Mazzola, J. Carrasquilla, M. Troyer, R. Melko, and G. Carleo, “Neural-network quantum state tomography,” *Nature Physics* **14**, 447 (2018).
- [28] J. Carrasquilla, G. Torlai, R. G. Melko, and L. Aolita, “Reconstructing quantum states with generative models,” *Nat. Mach. Intell.* **1**, 155 (2019).
- [29] D.-L. Deng, “Machine learning detection of bell nonlocality in quantum many-body systems,” *Phys. Rev. Lett.* **120**, 240402 (2018).
- [30] Y. Zhang and E.-A. Kim, “Quantum loop topography for machine learning,” *Phys. Rev. Lett.* **118**, 216401 (2017).
- [31] J. Carrasquilla and R. G. Melko, “Machine learning phases of matter,” *Nature Physics* **13**, 431 (2017).
- [32] E. P. van Nieuwenburg, Y.-H. Liu, and S. D. Huber, “Learning phase transitions by confusion,” *Nat. Phys.* **13**, 435 (2017).
- [33] L. Wang, “Discovering phase transitions with unsupervised learning,” *Phys. Rev. B* **94**, 195105 (2016).
- [34] P. Broecker, J. Carrasquilla, R. G. Melko, and S. Trebst, “Machine learning quantum phases of matter beyond the fermion sign problem,” *Sci. Rep.* **7** (2017), [10.1038/s41598-017-09098-0](#).
- [35] K. Ch’ng, J. Carrasquilla, R. G. Melko, and E. Khatami, “Machine learning phases of strongly correlated fermions,” *Phys. Rev. X* **7**, 031038 (2017).
- [36] Y. Zhang, R. G. Melko, and E.-A. Kim, “Machine learning  $z_2$  quantum spin liquids with quasiparticle statistics,” *Phys. Rev. B* **96**, 245119 (2017).
- [37] S. J. Wetzel, “Unsupervised learning of phase transitions: From principal component analysis to variational autoencoders,” *Phys. Rev. E* **96**, 022140 (2017).
- [38] W. Hu, R. R. P. Singh, and R. T. Scalettar, “Discovering phases, phase transitions, and crossovers through unsupervised machine learning: A critical examination,” *Phys. Rev. E* **95**, 062122 (2017).
- [39] Y. Zhang, A. Mesaros, K. Fujita, S. Edkins, M. Hamidian, K. Ch’ng, H. Eisaki, S. Uchida, J. S. Davis, E. Khatami, *et al.*, “Machine learning in electronic-quantum-matter imaging experiments,” *Nature* **570**, 484 (2019).
- [40] W. Lian, S.-T. Wang, S. Lu, Y. Huang, F. Wang, X. Yuan, W. Zhang, X. Ouyang, X. Wang, X. Huang, L. He, X. Chang, D.-L. Deng, and L. Duan, “Machine learning topological phases with a solid-state quantum simulator,” *Phys. Rev. Lett.* **122**, 210503 (2019).
- [41] M. Sewak, *Deep Reinforcement Learning: Frontiers of Artificial Intelligence* (Springer, 2019).
- [42] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of go with deep neural networks and tree search,” *Nature* **529**, 484 (2016).
- [43] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, “Solving

- the rubik's cube with deep reinforcement learning and search," *Nature Machine Intelligence* **1**, 356 (2019).
- [44] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *Nature* **518**, 529 (2015).
  - [45] M. L. Puterman and M. C. Shin, "Modified policy iteration algorithms for discounted markov decision problems," *Management Science* **24**, 1127 (1978).
  - [46] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics* **4**, 100 (1968).
  - [47] I. Pohl, "Heuristic search viewed as path finding in a graph," *Artificial intelligence* **1**, 193 (1970).
  - [48] See Supplemental Material at [URL will be inserted by publisher] for details on the structure of the neural networks, the grid search, the pseudocodes for the algorithm, and more numerical data, which includes Refs. [70–73].
  - [49] C. Nayak, S. H. Simon, A. Stern, M. Freedman, and S. Das Sarma, "Non-abelian anyons and topological quantum computation," *Rev. Mod. Phys.* **80**, 1083 (2008).
  - [50] S. Das Sarma, M. Freedman, and C. Nayak, "Majorana zero modes and topological quantum computation," *npj Quantum Inf.* **1**, 15001 (2015).
  - [51] T. Karzig, Y. Oreg, G. Refael, and M. H. Freedman, "Universal geometric path to a robust majorana magic gate," *Phys. Rev. X* **6**, 031019 (2016).
  - [52] A. Y. Kitaev, "Fault-tolerant quantum computation by anyons," *Ann. Phys.* **303**, 2 (2003).
  - [53] A. Kitaev, "Anyons in an exactly solved model and beyond," *Ann. Phys.* **321**, 2 (2006).
  - [54] M. H. Freedman, M. Larsen, and Z. Wang, "A modular functor which is universal for quantum computation," *Communications in Mathematical Physics* **227**, 605 (2002).
  - [55] J. S. Xia, W. Pan, C. L. Vicente, E. D. Adams, N. S. Sullivan, H. L. Stormer, D. C. Tsui, L. N. Pfeiffer, K. W. Baldwin, and K. W. West, "Electron correlation in the second landau level: A competition between many nearly degenerate quantum phases," *Phys. Rev. Lett.* **93**, 176809 (2004).
  - [56] N. R. Cooper, N. K. Wilkin, and J. M. F. Gunn, "Quantum phases of vortices in rotating bose-einstein condensates," *Phys. Rev. Lett.* **87**, 120405 (2001).
  - [57] M. Freedman, C. Nayak, K. Shtengel, K. Walker, and Z. Wang, "A class of p,t-invariant topological phases of interacting electrons," *Ann. Phys.* **310**, 428 (2004).
  - [58] P. Fendley and E. Fradkin, "Realizing non-abelian statistics in time-reversal-invariant systems," *Phys. Rev. B* **72**, 024412 (2005).
  - [59] N. E. Bonesteel, L. Hormozi, G. Zikos, and S. H. Simon, "Braid topologies for quantum computation," *Physical review letters* **95**, 140503 (2005).
  - [60] H. Xu and X. Wan, "Constructing functional braids for low-leakage topological quantum computing," *Physical Review A* **78**, 042325 (2008).
  - [61] L. Hormozi, G. Zikos, N. E. Bonesteel, and S. H. Simon, "Topological quantum compiling," *Physical Review B* **75**, 165310 (2007).
  - [62] D.-L. Deng, C. Wu, J.-L. Chen, and C. Oh, "Fault-tolerant greenberger-horne-zeilinger paradox based on non-abelian anyons," *Physical review letters* **105**, 060402 (2010).
  - [63] M. Burrello, H. Xu, G. Mussardo, and X. Wan, "Topological quantum hashing with the icosahedral group," *Physical review letters* **104**, 160502 (2010).
  - [64] V. Kliuchnikov, A. Bocharov, and K. M. Svore, "Asymptotically optimal topological quantum compiling," *Physical review letters* **112**, 140504 (2014).
  - [65] C. Carnahan, D. Zeuch, and N. Bonesteel, "Systematically generated two-qubit anyon braids," *Physical Review A* **93**, 052328 (2016).
  - [66] D. Q. Huynh, "Metrics for 3d rotations: Comparison and analysis," *Journal of Mathematical Imaging and Vision* **35**, 155 (2009).
  - [67] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016) pp. 770–778.
  - [68] F. Vatan and C. Williams, "Optimal quantum circuits for general two-qubit gates," *Physical Review A* **69**, 032315 (2004).
  - [69] D. Dong and I. R. Petersen, "Quantum control theory and applications: a survey," *IET Control Theory & Applications* **4**, 2651 (2010).
  - [70] A. L. Maas, A. Y. Hannun, and A. Y. Ng, "Rectifier nonlinearities improve neural network acoustic models," in *Proc. icml*, Vol. 30 (2013) p. 3.
  - [71] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv preprint arXiv:1502.03167* (2015).
  - [72] B. Kraus and J. Cirac, "Optimal creation of entanglement using a two-qubit gate," *Physical Review A* **63**, 062309 (2001).
  - [73] R. R. Tucci, "An introduction to cartan's kak decomposition for qc programmers," *arXiv preprint quant-ph/0507171* (2005).

## Supplementary Material

In this Supplementary Material, we provide more details for our algorithm introduced in the main text and more numerical data about applying this algorithm to compiling Fibonacci anyons.

*Neural network architecture.*—Our deep neural network (DNN) consists of two hidden layers, six residual blocks [67], and one output neuron. The first two hidden layers are of sizes 5000 and 1000, respectively, and each residual block consists of two hidden layers with 1000 hidden neurons each. The activation function is leaky ReLU [70] throughout the neural network, and batch normalization [71] is applied in all layers.

*Training the DNN.*—The DNN is trained to approximate the cost-to-go function  $J(s)$  of the input unitary matrix. During training, the DNN only utilized the following knowledge [43]:

1.  $J(s_0) = 0$  for the identity matrix  $s_0$ .
2. For an arbitrary state  $s$  other than  $s_0$ ,

$$J(s) = \min_a (J(S(s, a))) + 1 \quad (4)$$

where  $S(s, a)$  is the state obtained after applying the action  $a$  to the state  $s$ . In other words, the cost-to-go for each state equals to the cost-to-go of its predecessor plus the cost for each action.

Starting from the identity matrix  $s_0$  and iterate with the two rules above, the cost-to-go function for every matrix in the state space is uniquely determined. In practice, we set  $J(s) = 0$  for all the  $s$  with  $d(s, s_0) < 10^{-4}$ , where  $d(a, b)$  is the quaternion distance [66] defined in the main text.

Before training, the DNN is initialized with random small weights, and its initial guess for  $J(s)$  will be random small numbers. To train the DNN, we feed matrices generated by random gate sequences into the DNN, and optimize the DNN weights by minimizing the loss function  $\sum_s [J(s) - \min_a (J(S(s, a))) + 1]^2$  with the standard back-propagation algorithm.

To ensure that the DNN gradually explores the state space, we generate the training data by applying  $k$  random operations to the identity matrix, where  $k$  is evenly distributed between 1 and a maximum length  $M$ . We set  $M = 5$  in the beginning, and increases  $M$  by 1 whenever the loss falls below a predefined threshold  $\delta$ .

Training a neural network with itself as its target causes instabilities. In the machine learning community, the standard technique to overcome this issue is to use two neural networks during training [43]: a policy network and a target network. These two neural networks are almost identical, except that only the policy network is being trained with back-propagation. The target network stays intact most of the time and provides estimations of the updated cost-to-go function in Eq. (4). Whenever loss falls below  $\delta$ , the target network will be updated by copying parameters from the policy network.

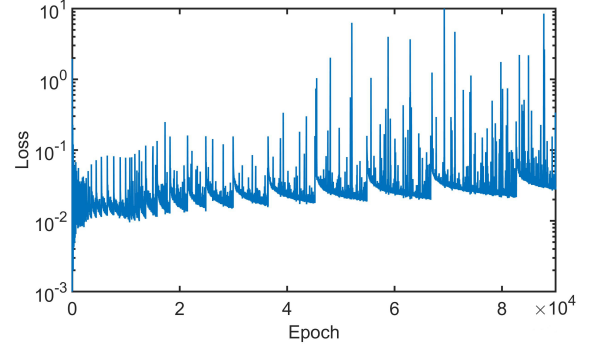


FIG. 5. The loss curve during training. The loss function spikes up whenever we raise the maximum length  $M$ , then training gradually brings the loss back until it falls below the threshold  $\delta$ , after which we raise  $M$  again. Here and throughout this whole Supplementary Material, the data plotted are all based on topological compiling with Fibonacci anyons.

The training algorithm is summarized in Algorithm 1.

Fig. 5 shows the loss curve during training. For reinforcement learning, large fluctuations in the loss function are common: whenever the DNN sees new data or updates the target network, the loss spikes up. Nevertheless, the performance of the DNN is stable. Fig. 6 shows the average accuracy  $\bar{\epsilon}$  of the gate sequences generated by our DNN at different training stages, in which  $D_{\max} = 100$  and each data point is calculated by averaging loss over 100 sample instances.  $\bar{\epsilon}$  saturated after  $2 \times 10^4$  epochs of training, and did not improve much afterwards. We trained this model on a single NVIDIA TITAN V GPU for about a week, but the result indicates that two days' training is already sufficient.

*Training data generation.*—To train the DNN, we need unitary matrices generated by random gate sequences, but a vast amount of data is required. To generate gate sequences more efficiently, we first performed a brute-force expansion, evaluating all possible gate sequences to a predefined brute-force depth. After that, further operations are generated randomly, until the length of the gate sequence reaches the predefined length  $k$ . During this procedure, we store all the encountered

---

### Algorithm1 Training the DNN

---

**Output:** The trained DNN parameters  $\theta_p$   
Initialize the polynet parameters  $\theta_p$   
Targetnet parameters  $\theta_t \leftarrow \theta_p$   
**for**  $i = 1$  to max\_epoch **do**  
Generate training data  $s$   
 $\text{loss} \leftarrow \sum_s (J_p(s) - \min_a (J_t(S(s, a))) + 1)^2$   
loss.backward()  
optimizer.step()  
**if**  $\text{loss} < \delta$  **then**  
 $\theta_t \leftarrow \theta_p$   
Maximum length  $M \leftarrow M + 1$   
**end if**  
**end for**

---

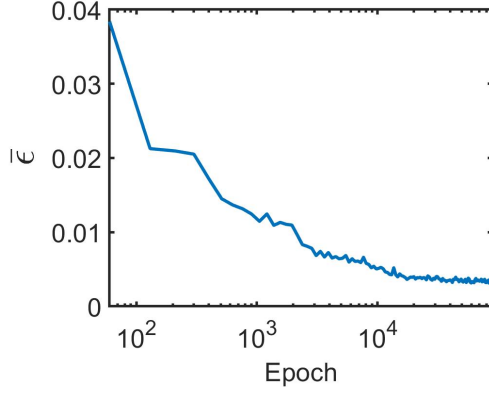


FIG. 6. The performance of the DNN during different training stages, evaluated as the averaged approximation accuracy  $\bar{\epsilon}$  of gate sequence solutions for 100 randomly generated SU(2) unitaries. The searching depth  $D_{\max} = 100$ .

matrices in the memory for later training, and the brute-force expansion depth is adjusted according to available memories.

We mention that the generation of training data in this way is very efficient since its complexity is mainly about two-by-two matrix multiplications. In fact, the whole data generation procedure is carried out in parallel on a single GPU, and takes only less than one second. The matrices generated by random sequences are periodically updated during training.

*Weighted A\* search with a decimal penalty term.*—A\* search [46] is a heuristic searching algorithm that finds a path between an initial node  $s_i$  and a target node  $s_0$  on a graph. The evaluation function that A\* search employs is defined as  $f(s) = G(s) + J(s)$ , where  $G(s)$  is the path cost (the cost already taken to reach  $s$  from  $s_i$ ), and  $J(s)$  is a heuristic function that estimates the cost still required to reach  $s_0$  from  $s$ , or the cost-to-go function. During the search, the A\* algorithm maintains a set of intermediate states  $\{s\}$ . The algorithm starts

---

**Algorithm2** Generating gate sequence with A\* search

---

**Input:** The target gate to be approximated,  $T$

**Output:** A gate sequence  $U_1 U_2 \dots U_n \approx T$

```

{ $s$ }  $\leftarrow$  { $T$ }
for  $i = 1$  to  $D_{\text{bf}}$  do
  { $s$ }  $\leftarrow$  the universal gate set  $\{U_i\} \times \{s\}$ 
  ( $\times$  denotes Cartesian product)
  Update and store  $s_{\text{best}}$ , the matrix closest to identity
end for
for  $i = 1$  to  $D_{\max}$  do
  Choose  $N$  states { $S$ } with minimum  $f(s)$  from { $s$ }
  { $S'$ }  $\leftarrow$  { $U_i$ }  $\times$  { $S$ }
  Update  $s_{\text{best}}$ 
  Add { $S'$ } to { $s$ } and remove { $S$ }
  if |{ $s$ }| > maximum_size then
    Remove the states with maximum  $f(s)$ 
  end if
end for
Return  $s_{\text{best}}$  and its corresponding gate sequence
Inverse the gate sequence

```

---

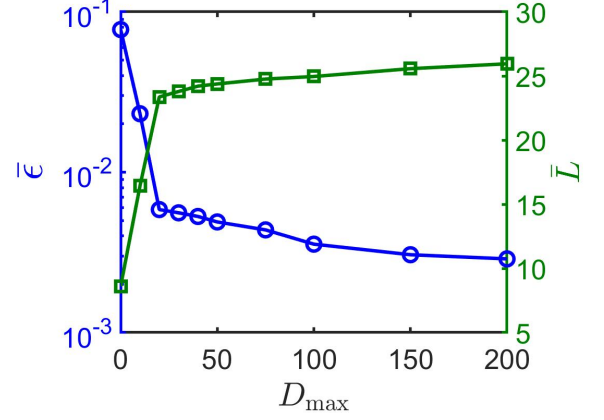


FIG. 7. We search for the optimal searching depth  $D_{\max}$  by plotting the average accuracy  $\bar{\epsilon}$  and the average braiding sequence length  $\bar{L}$  against  $D_{\max}$ , and choose  $D_{\max} = 100$  to balance between approximation accuracy and time cost. Higher accuracy can be achieved by increasing  $D_{\max}$  at the cost of computation time.

with only  $s_i$  in  $\{s\}$ . In each iteration, the algorithm selects and removes from  $\{s\}$  the  $s$  with the smallest  $f(s)$ , and adds in its successors if they have never been in  $\{s\}$  previously. This procedure repeats until the target node  $s_0$  appears in  $\{s\}$ .

It is proved that A\* search can find the shortest possible path if  $J(s)$  never overestimates the actual cost [46]. However, as A\* search stores all the nodes it generates, in general, it has an exponential space complexity, making it unpractical on large graphs. Weighted A\* search [47] partially resolves this issue by defining the evaluation function as  $f(s) = \lambda G(s) + J(s)$ , where  $\lambda$  is a weighting factor between 0 and 1. This method trades potentially longer paths for potentially less memory usage, alleviating the memory burden on large graphs.

For our problem of quantum compiling, we have an infinite graph in a continuous state space, and A\* search may face convergence issues. To resolve this, we imposed a maximum searching depth  $D_{\max}$ : searching terminates after reaching  $D_{\max}$  and the state closest to the target state is returned. Since the DNN heuristic  $J(s)$  can be non-reliable in extreme cases, as explained in the main test, we added an additional decimal penalty term. The evaluation function reads:

$$f(s) = \lambda G(s) + J(s) + \gamma \frac{(J(s) - \text{round}(J(s)))^2}{J(s)} \quad (5)$$

In order to better utilize the parallelization of GPU, we add a short brute-force search phase in the beginning, starting with the initial matrix and apply all possible gate sequences onto it up to a length of  $D_{\text{bf}}$ , storing all encountered states. Using the states generated by brute-force search as the initial set of intermediate states  $\{s\}$ , we evaluate their  $f(s)$  in parallel and expand  $N$  states with the smallest  $f(s)$ , removing them from  $\{s\}$  while adding their successors in, and so on so forth. Whenever the size of  $\{s\}$  reaches a maximum value, the states with

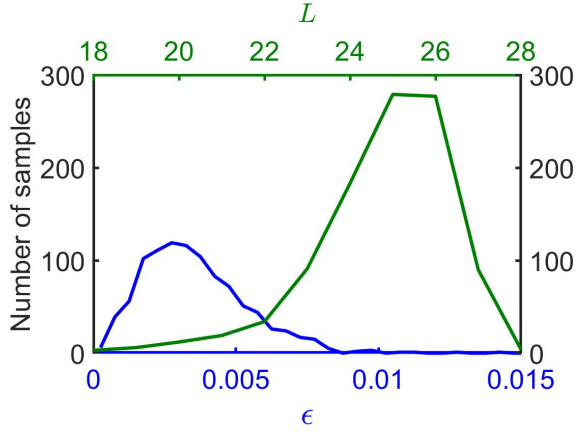


FIG. 8. The distribution of the approximation accuracy  $\epsilon$  and the sequence length  $L$  on the solution of 1000 randomly generated  $SU(2)$  matrices using our reinforcement learning algorithm, where the searching depth  $D_{\max} = 100$ . Note that the distribution of  $\epsilon$  has a long tail, while the distribution of  $L$  is more close to a Gaussian distribution. To better characterize  $\bar{\epsilon}$ , we adopt the typical average  $\bar{\epsilon} = \exp(\log \epsilon_i)$ . In this figure,  $\bar{\epsilon} = 0.0031$ , and  $\bar{L} = 24.79$ .

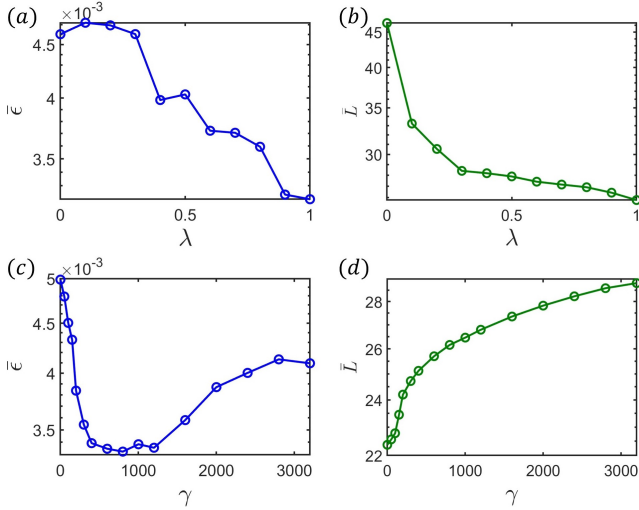


FIG. 9. (a) The average accuracy  $\bar{\epsilon}$  and (b) average sequence lengths  $\bar{L}$  versus  $\lambda$  for  $\gamma = 400$ .  $\lambda = 1$  leads to the shortest sequence and the highest accuracy. (c) The average accuracy  $\bar{\epsilon}$  and (d) average sequence lengths  $\bar{L}$  versus  $\gamma$  for  $\lambda = 1$ . The accuracy peaked around  $\gamma \sim 1000$ , while the sequence length increases monotonically with  $\gamma$ .

the largest  $f(s)$  are discarded. The procedure terminates after the number of searching steps reaches the maximum searching depth  $D_{\max}$ , upon which the gate sequence closest to the target state is returned. The searching algorithm is summarized in Algorithm 2. In the main text, we also have our algorithm terminate once a state with accuracy  $\epsilon < \epsilon_T$  is found. Together with a larger  $D_{\max}$  that is no longer a limiting factor, we can reach the desired accuracy and analyze the time and length complexity of our algorithm.

The computation time required for each searching step is

roughly a constant, and  $D_{\text{bf}}$  is small so that the time for the initial brute-force search is negligible. Therefore,  $D_{\max}$  is directly proportional to the time cost and responsible for the time complexity of our algorithm. Fig. 7 shows how the average accuracy  $\bar{\epsilon}$  and sequence length  $\bar{L}$  change with  $D_{\max}$ . We can see that the accuracy improves dramatically at  $D_{\max} < 30$  and starts to saturate after  $D_{\max} \sim 100$ , while the sequence length also reaches a plateau after  $D_{\max} \sim 50$ . Considering time efficiency, we set  $D_{\max} = 100$ , which roughly takes three seconds on a single GPU. Higher accuracy can be achieved by further increasing  $D_{\max}$  at the cost of computation time.

Fig. 8 shows the distributions of the approximation accuracy  $\epsilon$  and the sequence length  $L$  on the solution of 1000 randomly generated unitaries with  $D_{\max} = 100$ . Note that the distribution of  $\epsilon$  has a long tail. Therefore, to better characterize  $\bar{\epsilon}$ , we adopt the typical average  $\bar{\epsilon} = \exp(\log \epsilon_i)$ , which is  $\bar{\epsilon} = 0.0031$ , and  $\bar{L} = 24.79$  in this figure.

The parameters  $\lambda$  and  $\gamma$  in  $f(s)$  are determined with a grid search. In weighted  $A^*$  search,  $\lambda < 1$  is a compromise between insufficient memory and optimal paths. Since we manually imposed a maximum searching depth  $D_{\max}$ , memory is no longer a limiting issue for us, and  $\lambda = 1$  should yield the best results. Indeed, Fig. 9 (a)(b) verifies that  $\lambda = 1$  leads to the shortest sequence and the highest accuracy. Note that for longer searching depth  $D_{\max}$  and more complex scenarios, the optimal  $\lambda$  may become different. Fig. 9 (c)(d) shows the average accuracy  $\bar{\epsilon}$  and the average sequence length  $\bar{L}$  versus  $\gamma$  for  $\lambda = 1$ . The average accuracy first decreases and then increases as  $\gamma$  increases, while the sequence length increases monotonically with  $\gamma$ . We set  $\gamma = 400$  to balance between accuracy and sequence length.

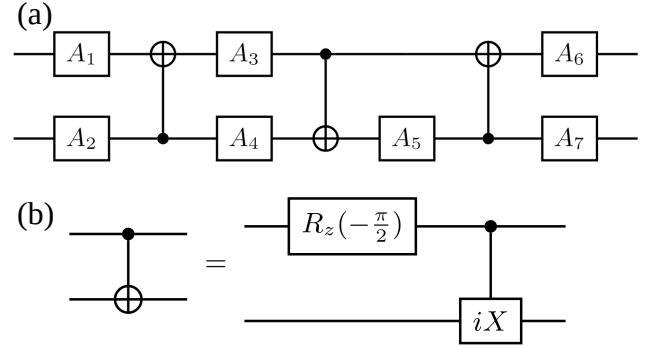


FIG. 10. (a) The quantum circuit can decompose an arbitrary two-qubit gate in  $SU(4)$  into seven one-qubit gates and three CNOT gates, up to a global phase [68]. The details of the implementation are available in [68, 72, 73]. (b) A CNOT gate can be decomposed into a controlled- $iX$  gate and a single-qubit rotation  $R_z(-\frac{\pi}{2})$ . By replacing the three CNOT gates with  $R_z(-\frac{\pi}{2}) + \text{controlled-}iX$  gate [59], merging the  $R_z(-\frac{\pi}{2})$  with the neighboring single-qubit unitaries and compiling them with our reinforcement learning algorithm, we reach our circuit for compiling an arbitrary two-qubit gate, in which every gate can be implemented by braiding Fibonacci anyons.

*Compiling two-qubit gates.*—As a second example, we

demonstrate how to decompose an arbitrary two-qubit gate into braidings of Fibonacci anyons with our algorithm.

To encode two qubits into Fibonacci anyons, we follow the conventions in [59] and represent them with two triplets, or six Fibonacci anyons. In principle, we can repeat our approach for single-qubit gates and train a more powerful DNN with an enlarged action space to compile an arbitrary two-qubit gate. However, the 87-dimensional Hilbert space for six Fibonacci anyons [59] is much larger than the three-dimensional Hilbert space for three anyons, which necessarily increases the training cost for the DNN. Unlike the single-qubit unitaries, we are unable to achieve a satisfactory accuracy for two-qubit unitaries with a single GPU within several days.

As an alternative, we can decompose every two-qubit gate into three CNOTs and seven single-qubit gates (Fig. 10a) [68]. This decomposition is analytical and proved optimal within its setup. In the meantime, the braiding sequence for the controlled- $iX$  gate is available [59, 61], which, when combined with a single-qubit rotation, offers the CNOT gate (Fig. 10b). The remaining single-qubit unitary components can be compiled using our reinforcement learning algorithm. Altogether, our algorithm can decompose an arbitrary two-qubit gate into braidings of Fibonacci anyons. Note that we have restricted particular gate sequences in the search by this construction scheme, which may impact the overall approximation accuracy and sequence length performance. However, as the degrees of freedom in multi-qubit gates scale exponentially, it may be advisable to include some extent of restriction and analytical decomposition on top of the more generic compiling by the reinforcement learning algorithm at lower level to balance the overall cost and performance. As we will see next, such a joint endeavor already makes a step forward in reducing the computational complexity in the two-qubit gate scenarios.

In Fig. 11, we compare the performance of the brute force search, the Solovay-Kitaev algorithm, and our reinforcement learning algorithm on compiling two-qubit gates. All three algorithms use the decomposition in Fig. 10, and the search algorithms are only responsible for the seven component single-qubit gates resulted from the decomposition. The controlled- $iX$  gate constructed in [59] has a braiding length of 140 and an error of  $2.7 \times 10^{-3}$  or  $1.8 \times 10^{-3}$  when the total charge of the five braiding anyons is  $\mathbf{I}$  or  $\tau$ , respectively. The leak-

age error, defined by the largest transition amplitude from a computational to non-computational state, is 0 when the total charge of the braiding anyons is  $\mathbf{I}$  and is  $1.4 \times 10^{-3}$  when the total charge is  $\tau$ . We evaluate the approximation accuracy with the 2-norm of the difference between the target and approximated gate.

In general, the approximation accuracy is very close irrespective of the total charge of the braiding anyons, and we plot them together as a single data point in Fig. 11. Importantly, the sequence lengths by the reinforcement learning algorithm and the brute force search both show a nearly linear scaling and are much smaller than the Solovay-Kitaev algorithm, indicating that similar to the single-qubit case in the main text,

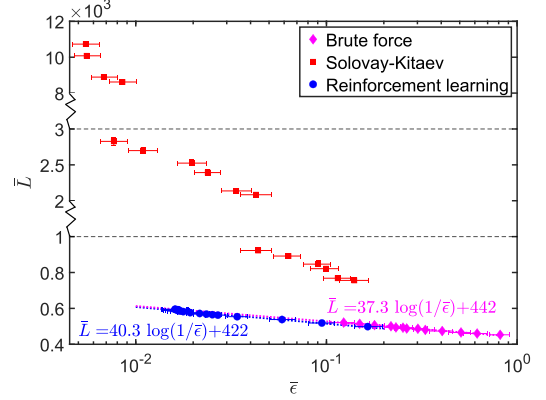


FIG. 11. Comparison between different algorithms on the length complexity of the generated braiding sequences when compiling two-qubit gates. All three algorithms use the decomposition in Fig. 10 and the controlled- $iX$  gate constructed in [59], and the searching algorithms are responsible for compiling the seven component single-qubit gates resulted from the decomposition. Each data point represents an average of 2000 trials (1000 each for the total charge of the braiding anyons being  $\mathbf{I}$  or  $\tau$ ) on randomly generated  $SU(4)$  matrices, and the error bars show the interval between the lower 25th and upper 75th percentile. Note the breaks in the vertical scale. Unlike Fig. 4 in the main text, we didn't perform the power-law fit, as the length and error of the approximated controlled- $iX$  gate interrupts the power-law scaling. Instead, we show that the sequence lengths of both the reinforcement learning algorithm and brute force search admit a linear fit, with reinforcement learning continuing the scaling for the brute force search.

the reinforcement learning algorithm also has an edge in two-qubit quantum compiling scenarios.